循序渐进，学习开发 xv6-riscv
第 5 章 第一个任务（task）

汪辰

目录

01

Hart 概述

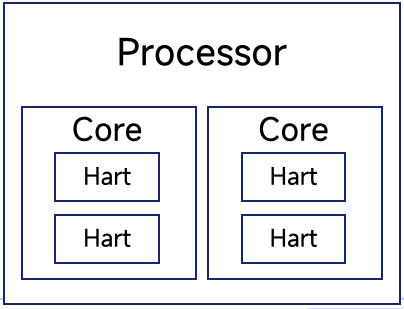
Hart 概述

Hart（HARdware Thread）中文可以叫做“**硬件线程**”。在 RISC-V 规范中，一个 hart 用于描述一个 **独立** 的执行单元。

- 每个 hart 都有自己 **独立** 的唯一标识 （mhartid）
- 每个 hart 都拥有自己 **独立** 的程序计数器（PC）、通用寄存器（x0-x31）和不同特权级别下的控制状态寄存器（CSR，如 mepc, sstatus）。
- RISC-V 特权规范（控制中断、异常、内存管理等）都是以 hart 为中心进行定义的。例如：每个 hart 都有自己 **独立** 的特权状态（M-mode、S-mode 等）；每个 hart **独立** 处理自己的中断和异常；每个 hart 有自己 **独立** 的地址转换表（页表）。
- 每个 hart 可以 **独立** 地取指、译码、执行指令。

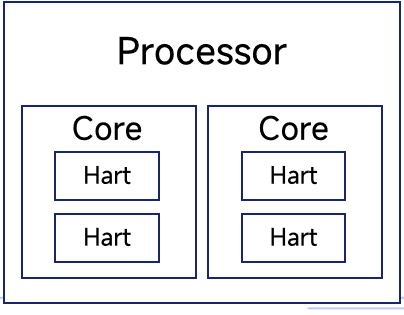
处理器拓扑与 Hart

- **处理器（Processor）**：插在主板系统插槽上的物理芯片。
- **核（Core）**：一个多核处理器上的一个独立的 CPU 实例。
- **硬件线程（Hardware Thread）**：一种在一个核上同时执行多个线程的 CPU 架构。又称为 SMT（Simulaneous MultiThreading）



处理器拓扑与 Hart

- **处理器（Processor）**：插在主板系统插槽上的物理芯片。
- **核（Core）**：一个多核处理器上的一个独立的 CPU 实例。
- **硬件线程（Hardware Thread）**：一种在一个核上同时执行多个线程的 CPU 架构。又称为 SMT（Simulaneous MultiThreading）

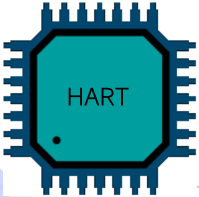



```
QEMUOPTS = -machine virt -bios none -kernel $K/kernel -m 128M -smp $(CPUS) -nographic
```

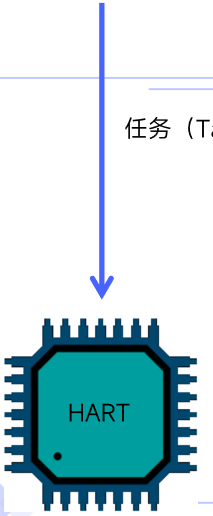
02

任务与上下文切换

任务

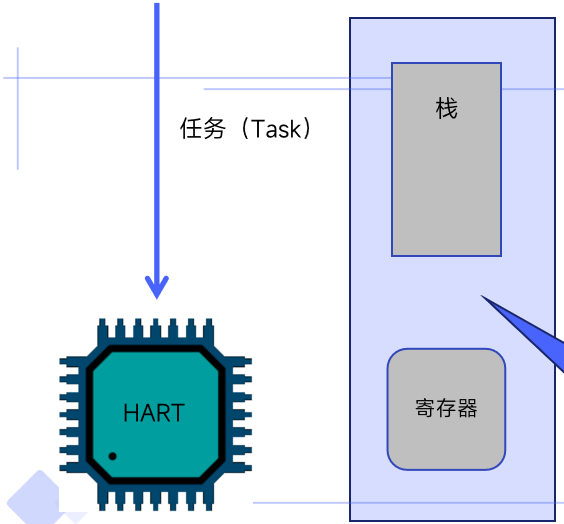


任务



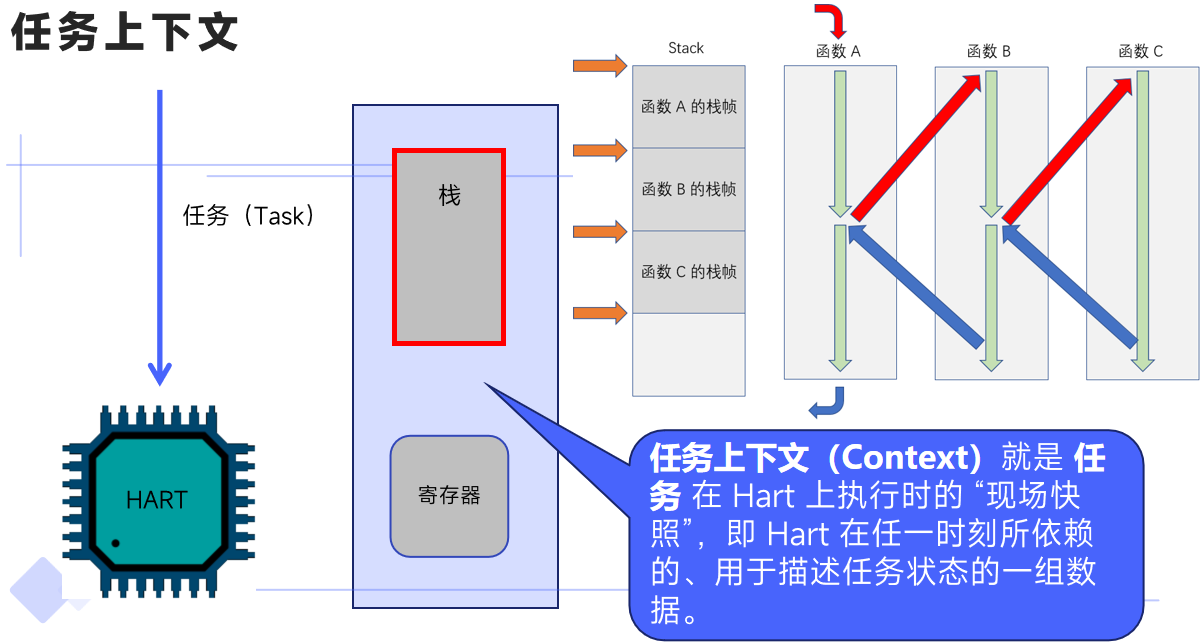
任务（Task） 是一个需要在 Hart 上运行的的指令执行流。也是操作系统需要管理的基本工作单元，也叫 **线程（Thread）** 。

任务上下文

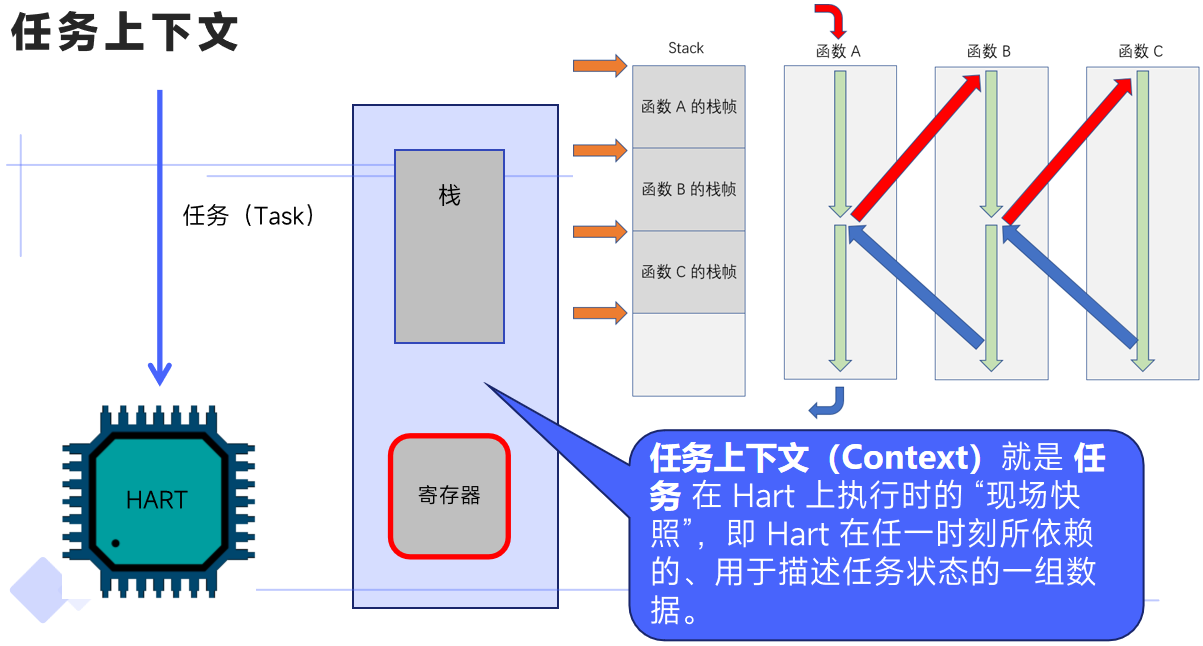


上下文（Context）就是 **任务** 在 Hart 上执行时的“现场快照”，即 Hart 在任一时刻所依赖的、用于描述任务状态的一组数据。

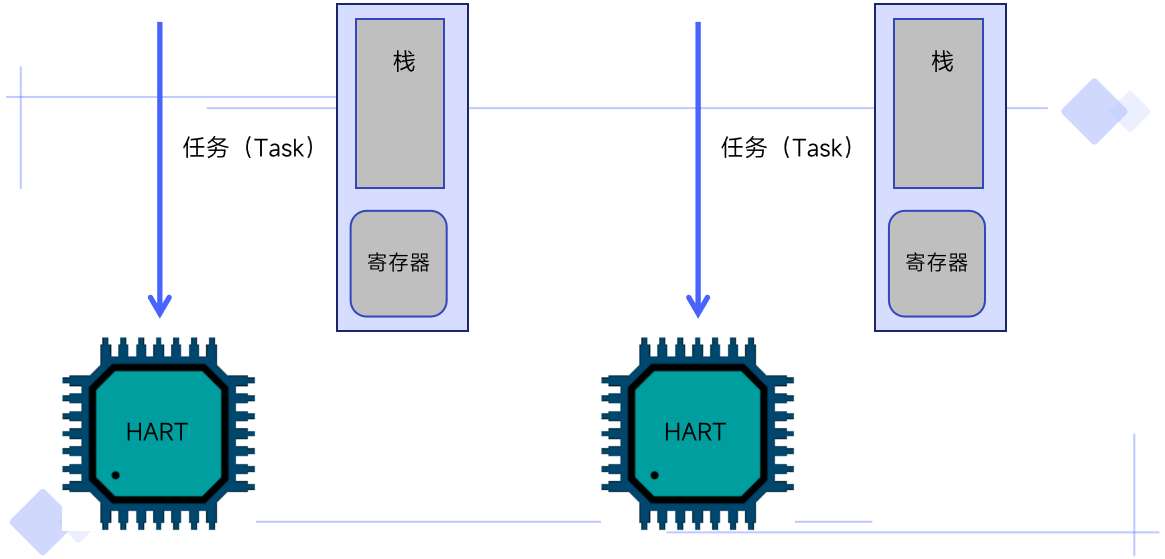
任务上下文


任务（Task）
HART
栈
寄存器
Stack
函数 A 的栈帧
函数 B 的栈帧
函数 C 的栈帧
函数 A
函数 B
函数 C
任务上下文（Context）就是 任务 在 Hart 上执行时的“现场快照”，即 Hart 在任一时刻所依赖的、用于描述任务状态的一组数据。

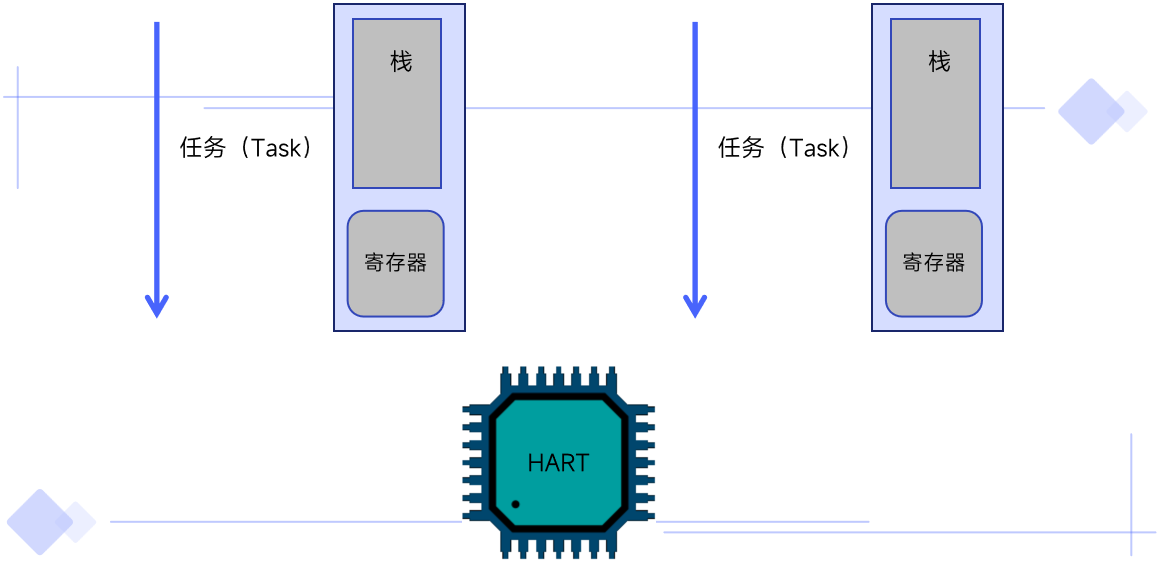
任务上下文


任务（Task）
HART
栈
寄存器
Stack
函数 A 的栈帧
函数 B 的栈帧
函数 C 的栈帧
函数 A
函数 B
函数 C
任务上下文（Context）就是 任务 在 Hart 上执行时的“现场快照”，即 Hart 在任一时刻所依赖的、用于描述任务状态的一组数据。

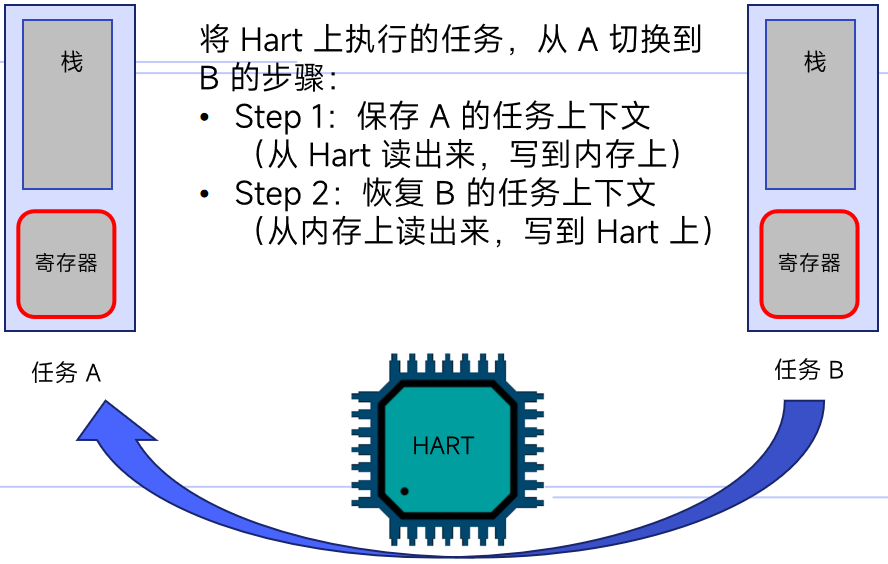
任务切换



任务切换



任务切换

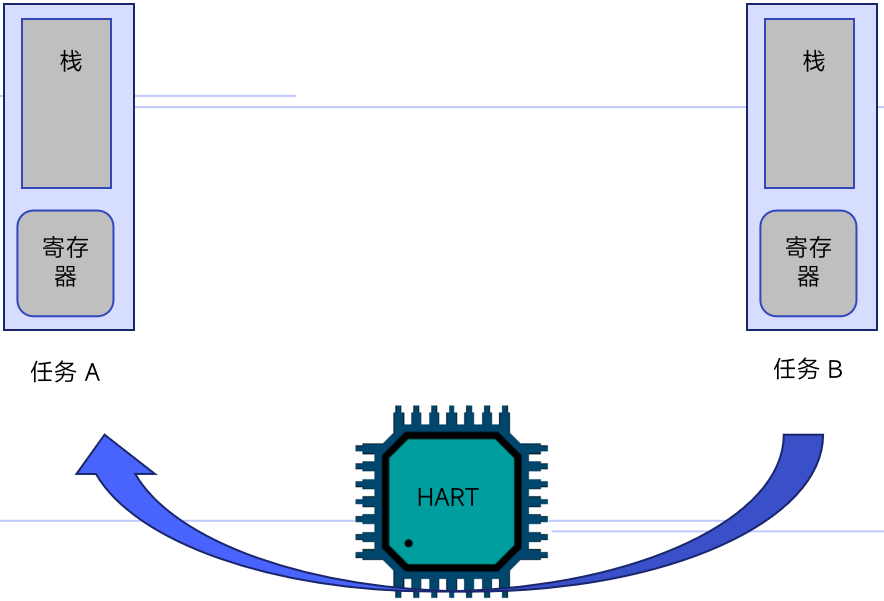


将 Hart 上执行的任务，从 A 切换到 B 的步骤：

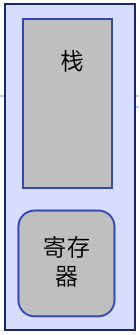
- Step 1：保存 A 的任务上下文
  （从 Hart 读出来，写到内存上）
- Step 2：恢复 B 的任务上下文
  （从内存上读出来，写到 Hart 上）

03 任务管理的设计与实现

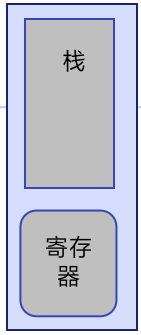
任务管理总体设计



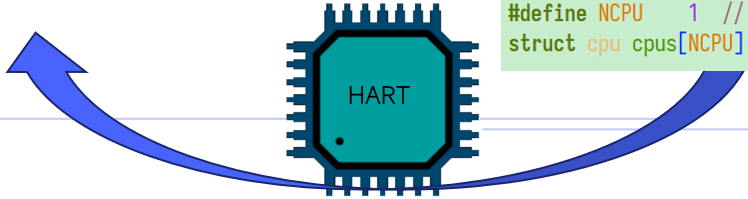
任务管理总体设计



任务 A

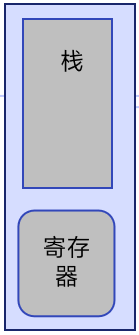


任务 B



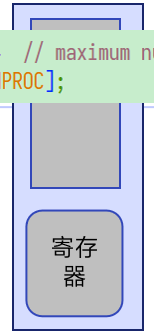

```
#define NCPU    1  // maximum number of CPUs
struct cpu cpus[NCPU];
```

任务管理总体设计

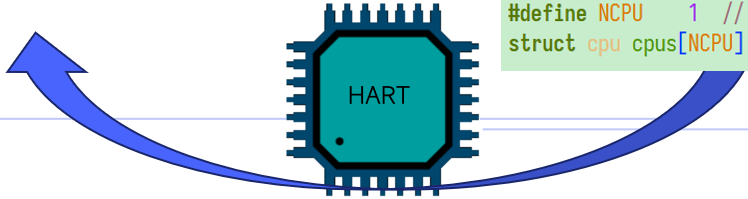


任务 A

```
#define NPROC   64  // maximum number of processes
struct proc proc[NPROC];
```

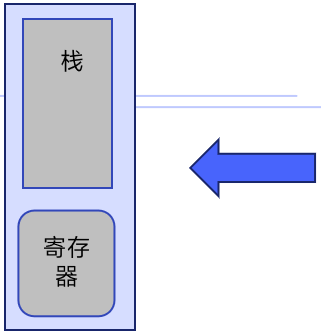


任务 B

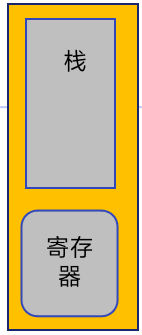



```
#define NCPU    1  // maximum number of CPUs
struct cpu cpus[NCPU];
```

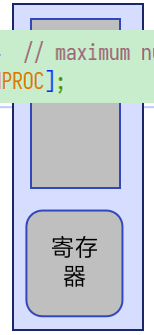
任务管理总体设计



任务 A

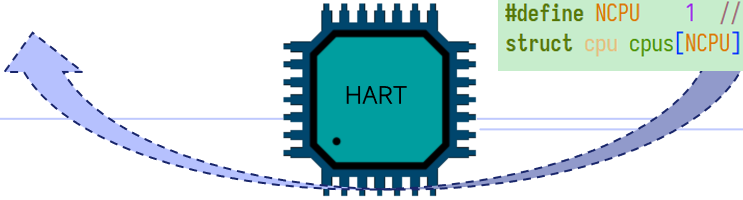


调度任务



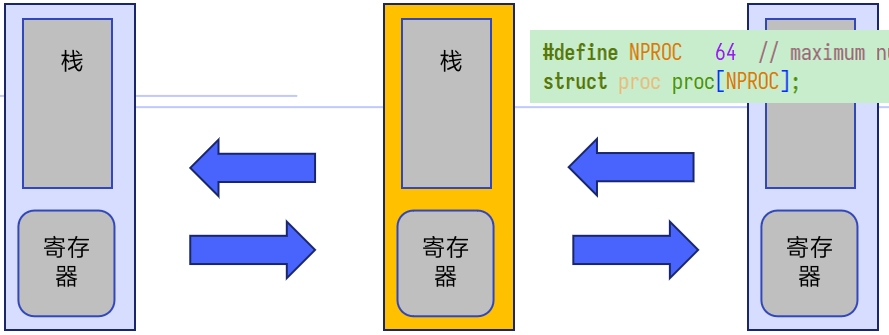
任务 B

```
#define NPROC   64  // maximum number of processes
struct proc proc[NPROC];
```

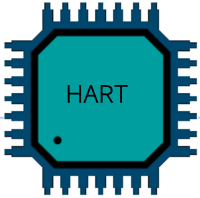



```
#define NCPU    1  // maximum number of CPUs
struct cpu cpus[NCPU];
```

任务管理总体设计



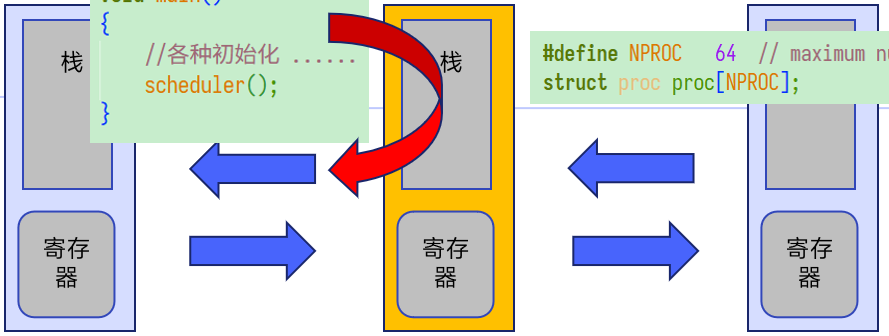

```
#define NPROC   64  // maximum number of processes
struct proc proc[NPROC];
```




```
#define NCPU    1  // maximum number of CPUs
struct cpu cpus[NCPU];
```

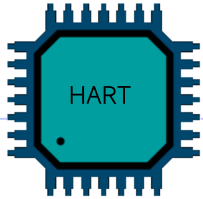
任务管理总体设计

```
void main()
{
    //各种初始化 ......
    scheduler();
}
```



任务 A　　调度任务　　任务 B

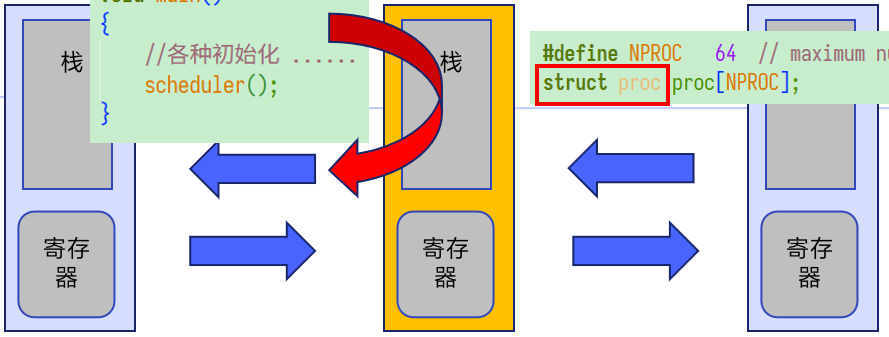
```
#define NPROC   64  // maximum number of processes
struct proc proc[NPROC];
```




```
#define NCPU    1  // maximum number of CPUs
struct cpu cpus[NCPU];
```

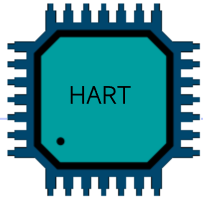
任务管理总体设计

```
void main()
{
    //各种初始化 ......
    scheduler();
}
```



任务 A　　调度任务　　任务 B

```
#define NPROC   64  // maximum number of processes
struct proc proc[NPROC];
```




```
#define NCPU    1  // maximum number of CPUs
struct cpu cpus[NCPU];
```

struct proc

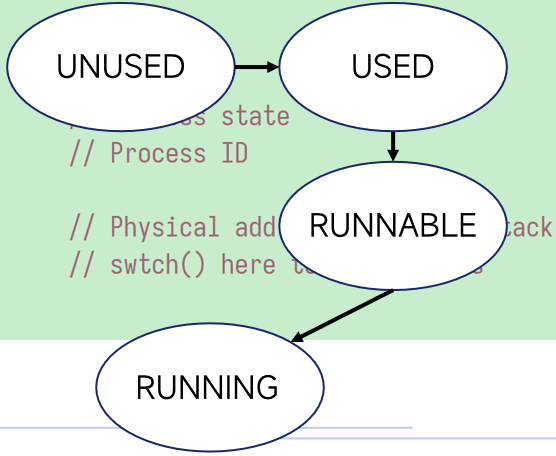
```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  enum procstate state;        // Process state
  int pid;                     // Process ID

  uint64 kstack;               // Physical address of kernel stack
  struct context context;      // swtch() here to run process
};
```

struct proc

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  enum procstate state;        // Process state
  int pid;                     // Process ID

  uint64 kstack;               // Physical address of kernel stack
  struct context context;      // swtch() here to run process
};
```

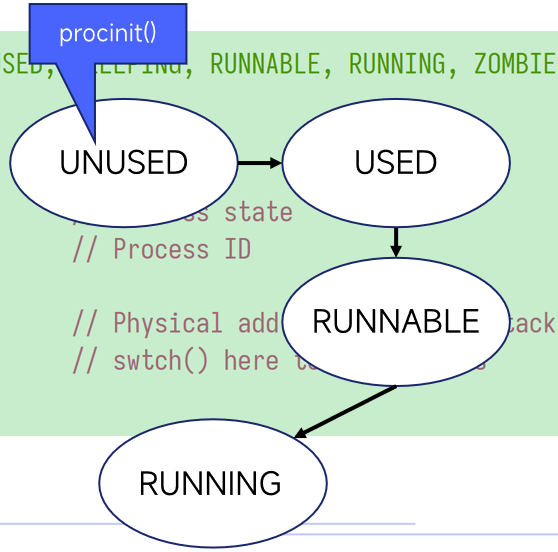
struct proc

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  enum procstate state;
  int pid;

  uint64 kstack;
  struct context context;
};
```



struct proc

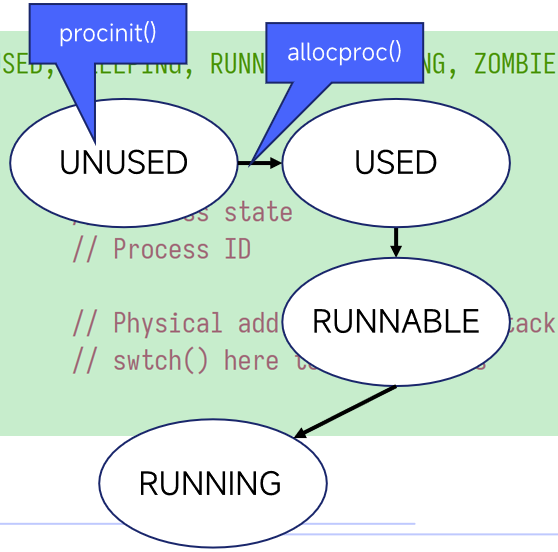
```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  enum procstate state;
  int pid;

  uint64 kstack;
  struct context context;
};
```



struct proc

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  enum procstate state;
  int pid;

  uint64 kstack;
  struct context context;
};
```

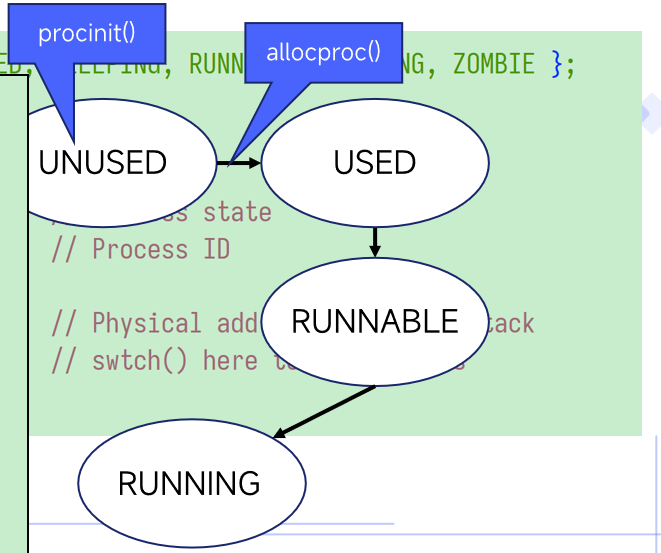
// Process state
// Process ID
// Physical address of kstack
// swtch() here to run process



struct proc

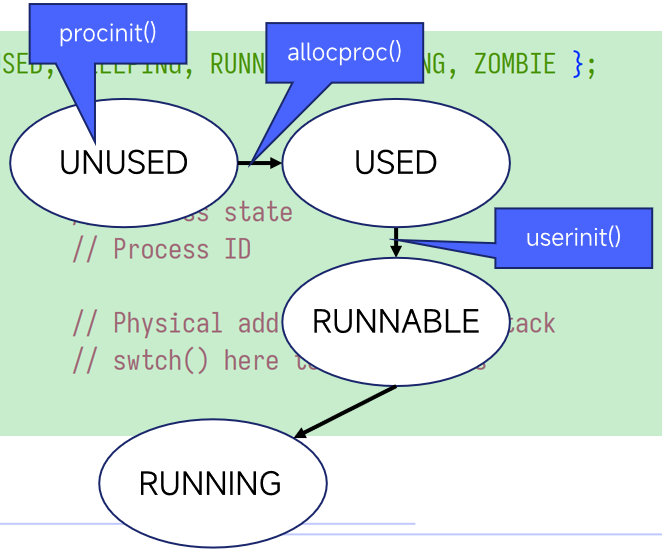
```
enum procstate { UNUSED, USED, ...LEEPING, RUNN...NG, ZOMBIE };
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  struct proc *p;
  for(p = proc; p < &proc[NPROC]; p++) {
    if(p->state == UNUSED) {
      goto found;
    }
  }
  return 0;
found:
  p->pid = allocpid();
  p->state = USED;
  // Set up new context to start executing
  ......
  return p;
}
```



struct proc

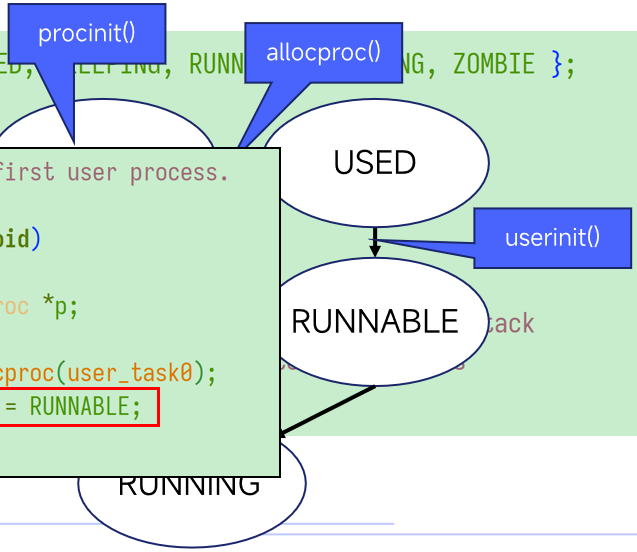
```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  enum procstate state;        // Process state
  int pid;                     // Process ID

  uint64 kstack;               // Physical address of kernel stack
  struct context context;      // swtch() here to run process
};
```



struct proc

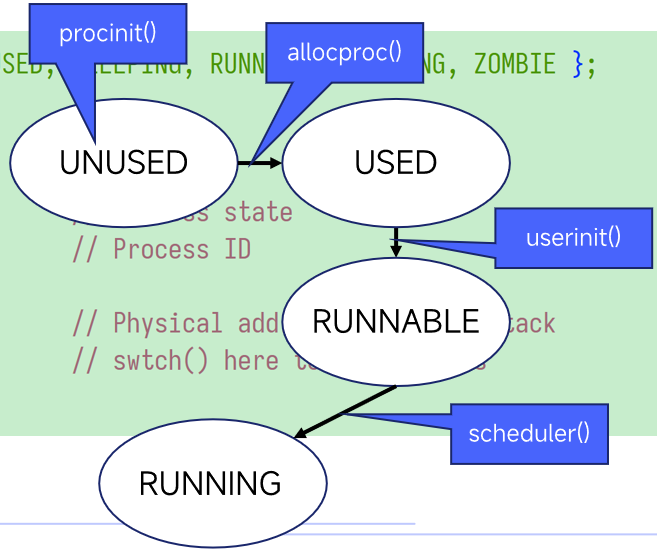
```
enum procstate { UNUSED, USED, ..., ZOMBIE };

// Per-process state
struct proc {
  enum procstate
  int pid;

  uint64 kstack;
  struct context
};
```

```
// Set up first user process.
void
userinit(void)
{
  struct proc *p;

  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```



struct proc

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  enum procstate state;        // Process state
  int pid;                     // Process ID

  uint64 kstack;               // Physical address of kernel stack
  struct context context;      // swtch() here to run process
};
```



struct proc

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  enum procstate state;        // Process state
  int pid;                     // Process ID

  uint64 kstack;               // Physical address of kernel stack
  struct context context;      // swtch() here to run process
};
```

struct proc

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process
struct proc {
  enum procstat                    state
  int pid;                         ID

  uint64 kstack                    address of kernel stack
  struct contex                    here to run process
};
```

```
int nextpid = 1;

int
allocpid()
{
  int pid;

  pid = nextpid;
  nextpid = nextpid + 1;

  return pid;
}
```

struct proc

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  enum procstate state;        // Process state
  int pid;                     // Process ID

  uint64 kstack;               // Physical address of kernel stack
  struct context context;      // swtch() here to run process
};
```

struct proc

```
enum procstate { UNU                                    BIE };

// Per-process state
struct proc {
  enum procstate sta
  int pid;

  uint64 kstack;                                        ack
  struct context con
};
```

```
// initialize the proc table.
void
procinit(void)
{
  struct proc *p;

  for(p = proc; p < &proc[NPROC]; p++) {
      p->state = UNUSED;
      p->kstack = (uint64)kalloc();
      if(p->kstack == 0)
          panic("procinit: kalloc");
  }
}
```

struct proc

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };

// Per-process state
struct proc {
  enum procstate state;        // Process state
  int pid;                     // Process ID

  uint64 kstack;               // Physical address of kernel stack
  struct context context;      // swtch() here to run process
};
```

struct proc

```
enum procstate { UNUSED, USED

// Per-process state
struct proc {
  enum procstate state;
  int pid;

  uint64 kstack;
  struct context context;
};
```

```
// Saved registers for kernel context switches.
struct context {
  uint64 ra;
  uint64 sp;

  // callee-saved
  uint64 s0;
  uint64 s1;
  uint64 s2;
  uint64 s3;
  uint64 s4;
  uint64 s5;
  uint64 s6;
  uint64 s7;
  uint64 s8;
  uint64 s9;
  uint64 s10;
  uint64 s11;
};
```

struct proc

```
enum procstate { UNUSED, USED

// Per-process state
struct proc {
  enum procstate state;
  int pid;

  uint64 kstack;
  struct context context;
};
```

```
// Saved registers for kernel context switches.
struct context {
  uint64 ra;
  uint64 sp;

  // callee-saved
  uint64 s0;
  uint64 s1;
  uint64 s2;
  uint64 s3;
  uint64 s4;
  uint64 s5;
  uint64 s6;
  uint64 s7;
  uint64 s8;
  uint64 s9;
  uint64 s10;
  uint64 s11;
};
```

struct cpu

```
// Per-CPU state.
struct cpu {
  struct proc *proc;          // The process running on this cpu, or null.
  struct context context;     // swtch() here to enter scheduler().
};
```

struct cpu

```
// Per-CPU state.
struct cpu {
  struct proc *proc;          // The process running on this cpu, or null.
  struct context context;     // swtch() here to enter scheduler().
};
```

struct cpu

```
// Per-CPU state.
struct cpu {
  struct proc *proc;          // The process running on this cpu, or null.
  struct context context;     // swtch() here to enter scheduler().
};
```

struct cpu

```
// Per-CPU state.
struct cpu {
  struct proc *proc;
  struct context context;
};
```

```
struct cpu cpus[NCPU];

int cpuid()
{
  return 0;
}

// Return this CPU's cpu struct.
struct cpu* mycpu(void)
{
  int id = cpuid();
  struct cpu *c = &cpus[id];
  return c;
}
```

struct cpu

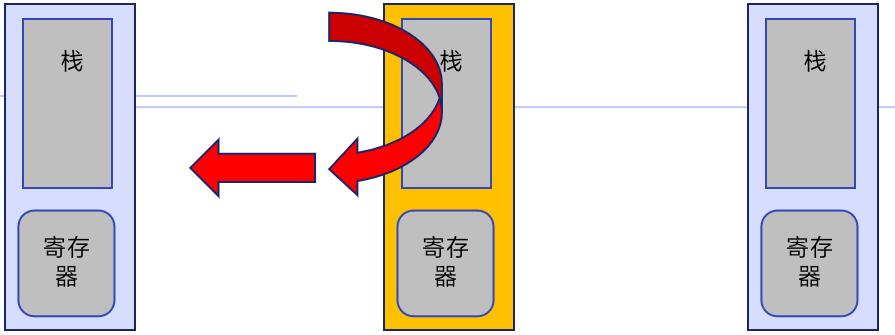
```
// Per-CPU state.
struct cpu {
  struct proc *proc;
  struct context context;
};
```

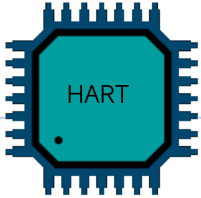
```
struct cpu cpus[NCPU];

int cpuid()
{
  return 0;
}

// Return this CPU's cpu struct.
struct cpu* mycpu(void)
{
  int id = cpuid();
  struct cpu *c = &
  return c;
}
```

```
int cpuid()
{
  return 0;
}
```

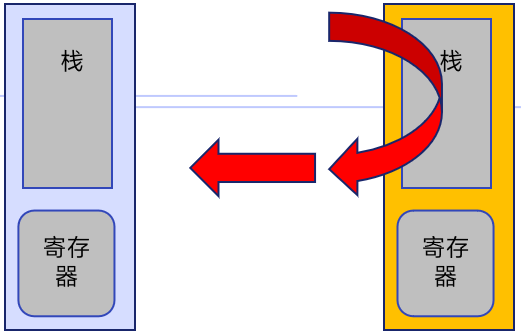
```
or null.
).
```

任务切换的实现

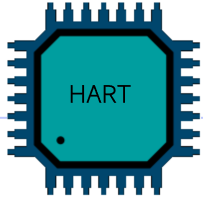




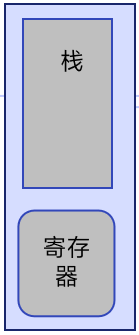
任务切换的实现




```
// start() jumps here in
// supervisor mode on all CPUs.
void main()
{
  // other initialization ......
  procinit(); // process table
  userinit(); // first user process
  scheduler();
}
```

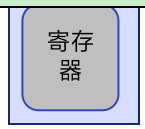


任务切换的实现



任务 A

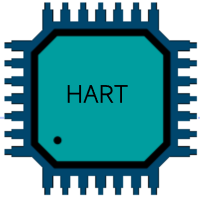
调度任务



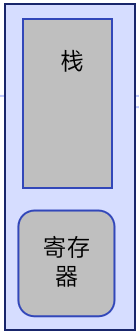
任务 B

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializations.....
  procinit(); // process table
  userinit(); // first user process
  scheduler();
}
```

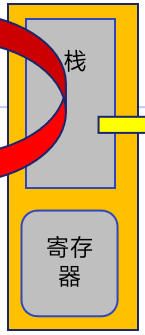
```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```



任务切换的实现



任务 A



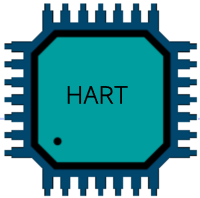
调度任务

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

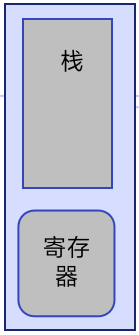
```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```


任务 B

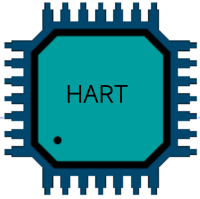


任务切换的实现



任务 A

调度任务




```c
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

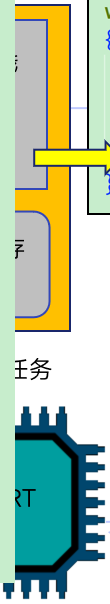
```c
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```c
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```c
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    for(p = proc; p < &proc[NPROC]; p++) {
      if(p->state == RUNNABLE) {
        // Switch to chosen process.
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```



```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    for(p = proc; p < &proc[NPROC]; p++) {
      if(p->state == RUNNABLE) {
        // Switch to chosen process.
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    for(p = proc; p < &proc[NPROC]; p++) {
      if(p->state == RUNNABLE) {
        // Switch to chosen process.
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    for(p = proc; p < &proc[NPROC]; p++) {
      if(p->state == RUNNABLE) {
        // Switch to chosen process.
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    for(p = proc; p < &proc[NPROC]; p++) {
      if(p->state == RUNNABLE) {
        // Switch to chosen process.
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

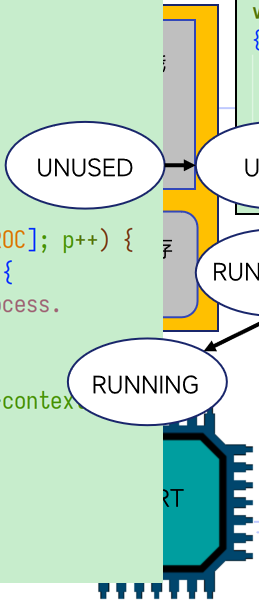
```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    for(p = proc; p < &proc[NPROC]; p++) {
      if(p->state == RUNNABLE) {
        // Switch to chosen process.
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->contex
        c->proc = 0;
      }
    }
  }
}
```




```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  init();
  er();
```

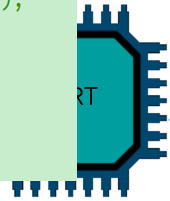
```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    for(p = proc; p < &proc[NPROC]; p++) {
      if(p->state == RUNNABLE) {
        // Switch to chosen process.
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```



```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    for(p = proc; p < &proc[NPROC]; p++) {
      if(p->state == RUNNABLE) {
        // Switch to chosen process.
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *
  struct cpu *c

  c->proc = 0;
  for(;;){
    for(p = pr
      if(p->sta
        // Swi
        p->sta
        c->proc
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
# void swtch(struct context *old,
#            struct context *new);
.globl swtch
swtch:
        sd ra, 0(a0)
        // save ......
        sd s11, 104(a0)
        ld ra, 0(a1)
        // restore ......
        ld s11, 104(a1)
        ret
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *
  struct cpu *c

  c->proc = 0;
  for(;;){
    for(p = pr
      if(p->st
        // Swi
        p->sta
        c->pro
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
# void swtch(struct context *ol
#            struct context *ne
.globl swtch
swtch:
        sd ra, 0(a0)
        // save ......
        sd s11, 104(a0)
        ld ra, 0(a1)
        // restore ......
        ld s11, 104(a1)
        ret
```

```
// Saved registers for kernel context switches.
struct context {
  uint64 ra;
  uint64 sp;

  // callee-saved
  uint64 s0;
  uint64 s1;
  uint64 s2;
  uint64 s3;
  uint64 s4;
  uint64 s5;
  uint64 s6;
  uint64 s7;
  uint64 s8;
  uint64 s9;
  uint64 s10;
  uint64 s11;
};
```

```
// start() jumps here in
oc();
return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *
  struct cpu *c

  c->proc = 0;
  for(;;){
    for(p = pr
      if(p->st
        // Swi
        p->sta
        c->proc
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
# void swtch(struct context *ol
#            struct context *ne
.globl swtch
swtch:
        sd ra, 0(a0)
        // save ......
        sd s11, 104(a0)
        ld ra, 0(a1)
        // restore ......
        ld s11, 104(a1)
        ret
```

```
// start() jumps here in
// Saved registers for kernel context switches.
struct context {
  uint64 ra;
  uint64 sp;

  // callee-saved
  uint64 s0;
  uint64 s1;
  uint64 s2;
  uint64 s3;
  uint64 s4;
  uint64 s5;
  uint64 s6;
  uint64 s7;
  uint64 s8;
  uint64 s9;
  uint64 s10;
  uint64 s11;
};
```

```
  return p;
}
```

Table 1. Integer register convention

Name	ABI Mnemonic	Meaning	Preserved across calls?
x0	zero	Zero	— (Immutable)
x1	ra	Return address	No
x2	sp	Stack pointer	Yes
x3	gp	Global pointer	— (Unallocatable)
x4	tp	Thread pointer	— (Unallocatable)
x5 - x7	t0 - t2	Temporary registers	No
x8 - x9	s0 - s1	Callee-saved registers	Yes
x10 - x17	a0 - a7	Argument registers	No
x18 - x27	s2 - s11	Callee-saved registers	Yes
x28 - x31	t3 - t6	Temporary registers	No

RISC-V ABIs Specification v1.0

任务切换的实现

```
void scheduler(void)
{
  struct proc
  struct cpu *

  c->proc = 0;
  for(;;){
    for(p = pr
      if(p->sta
        // Swi
        p->sta
        c->proc
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
# void swtch(struct context *old,
#            struct context *new);
.globl swtch
swtch:
        sd ra, 0(a0)
        // save ......
        sd s11, 104(a0)
        ld ra, 0(a1)
        // restore ......
        ld s11, 104(a1)
        ret
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc
  struct cpu *

  c->proc = 0;
  for(;;){
    for(p = pr
      if(p->sta
        // Swi
        p->sta
        c->proc
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
# void swtch(struct context *old,
#            struct context *new);
.globl swtch
swtch:
        sd ra, 0(a0)
        // save ......
        sd s11, 104(a0)
        ld ra, 0(a1)
        // restore ......
        ld s11, 104(a1)
        ret
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *
  struct cpu *c

  c->proc = 0;
  for(;;){
    for(p = pr
      if(p->sta
        // Swi
        p->sta
        c->proc
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
# void swtch(struct context *old,
#            struct context *new);
.globl swtch
swtch:
        sd ra, 0(a0)
        // save ......
        sd s11, 104(a0)
        ld ra, 0(a1)
        // restore ......
        ld s11, 104(a1)
        ret
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc*
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *
  struct cpu *c

  c->proc = 0;
  for(;;){
    for(p = pr
      if(p->sta
        // Swi
        p->sta
        c->proc
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
# void swtch(struct context *old,
#            struct context *new);
.globl swtch
swtch:
        sd ra, 0(a0)
        // save ......
        sd s11, 104(a0)
        ld ra, 0(a1)
        // restore ......
        ld s11, 104(a1)
        ret
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *
  struct cpu *c

  c->proc = 0;
  for(;;){
    for(p = pr
      if(p->sta
        // Swi
        p->sta
        c->proc
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
# void swtch(struct context *old,
#            struct context *new);
.globl swtch
swtch:
        sd ra, 0(a0)
        // save ......
        sd s11, 104(a0)
        ld ra, 0(a1)
        // restore ......
        ld s11, 104(a1)
        ret
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *
  struct cpu *c

  c->proc = 0;
  for(;;){
    for(p = pr
      if(p->sta
        // Swi
        p->sta
        c->proc
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
# void swtch(struct context *old,
#            struct context *new);
.globl swtch
swtch:
        sd ra, 0(a0)
        // save ......
        sd s11, 104(a0)
        ld ra, 0(a1)
        // restore ......
        ld s11, 104(a1)
        ret
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    for(p = proc; p < &proc[NPROC]; p++) {
      if(p->state == RUNNABLE) {
        // Switch to chosen process.  It is the process's job
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
# void swtch(struct context *old,
#            struct context *new);
.globl swtch
swtch:
        sd ra, 0(a0)
        // save ......
        sd s11, 104(a0)
        ld ra, 0(a1)
        // restore ......
        ld s11, 104(a1)
        ret
```

```
addi  a1,s1,16
mv    a0,s4
jal   ra, <swtch>
```

```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

任务切换的实现

```
void scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    for(p = proc; p < &proc[NPROC]; p++){
      if(p->state == RUNNABLE){
        // Switch to chosen process.  It is the process's job
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);
        c->proc = 0;
      }
    }
  }
}
```

```
# void swtch(struct context *old,
#            struct context *new);
.globl swtch
swtch:
        sd ra, 0(a0)
        // save ......
        sd s11, 104(a0)
        ld ra, 0(a1)
        // restore ......
        ld s11, 104(a1)
        ret
```

```
addi  a1,s1,16
mv    a0,s4
jal   ra, <swtch>
```

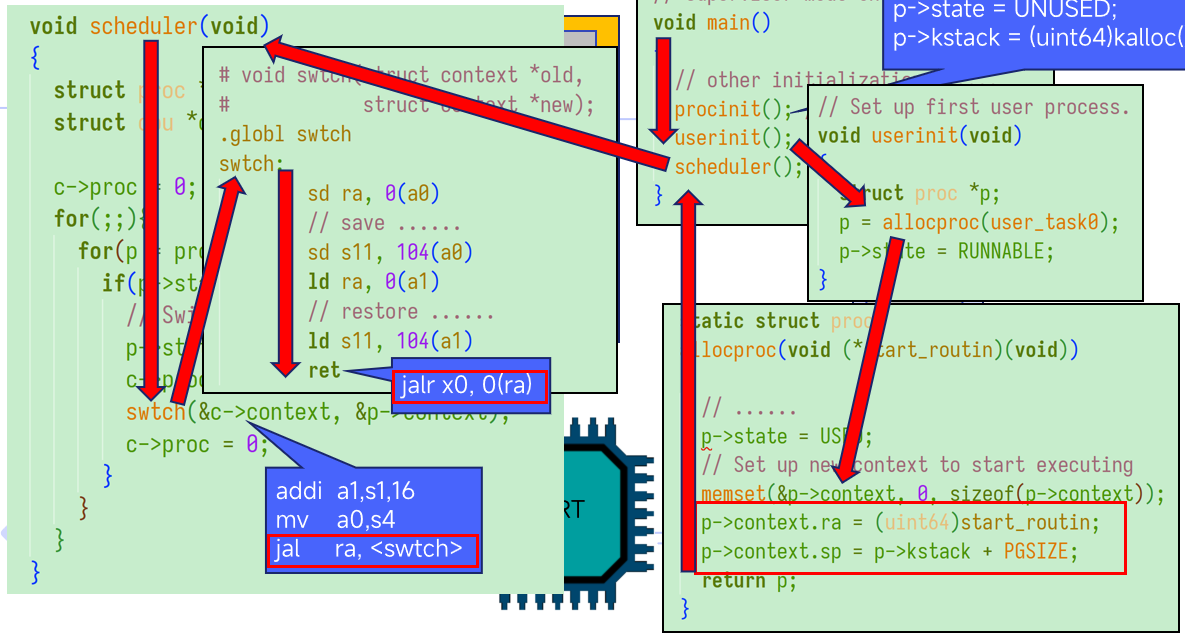
```
// start() jumps here in
// supervisor mode on
void main()
{
  // other initializati
  procinit();
  userinit();
  scheduler();
}
```

```
p->state = UNUSED;
p->kstack = (uint64)kalloc();
```

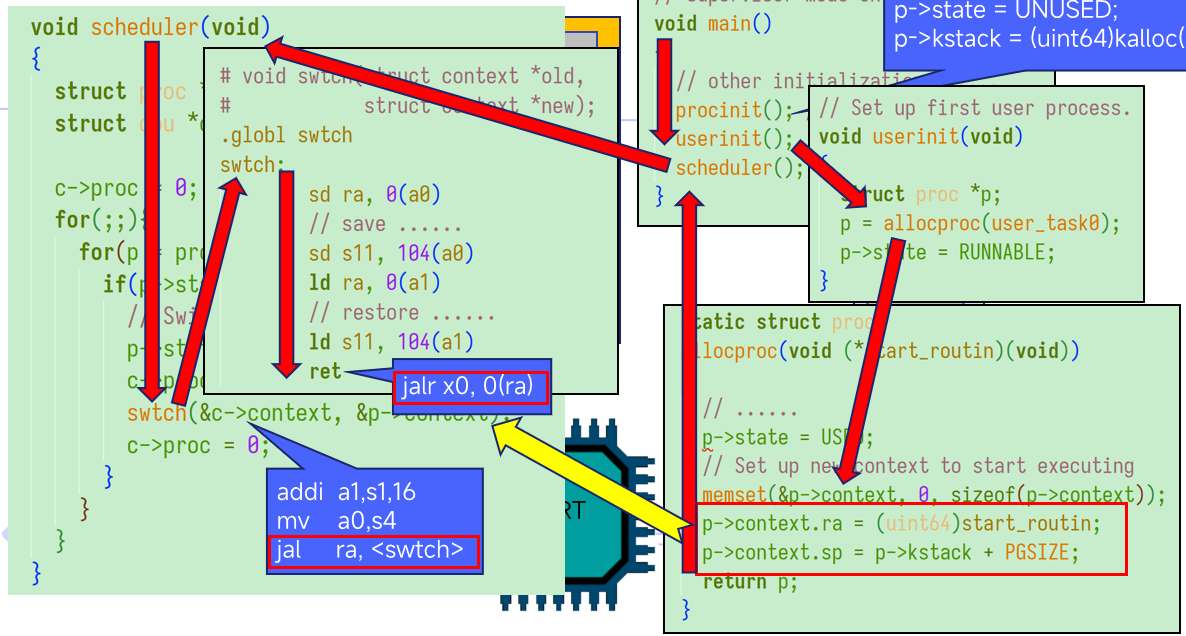
```
// Set up first user process.
void userinit(void)
{
  struct proc *p;
  p = allocproc(user_task0);
  p->state = RUNNABLE;
}
```

```
static struct proc
allocproc(void (*start_routin)(void))
{
  // ......
  p->state = USED;
  // Set up new context to start executing
  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)start_routin;
  p->context.sp = p->kstack + PGSIZE;
  return p;
}
```

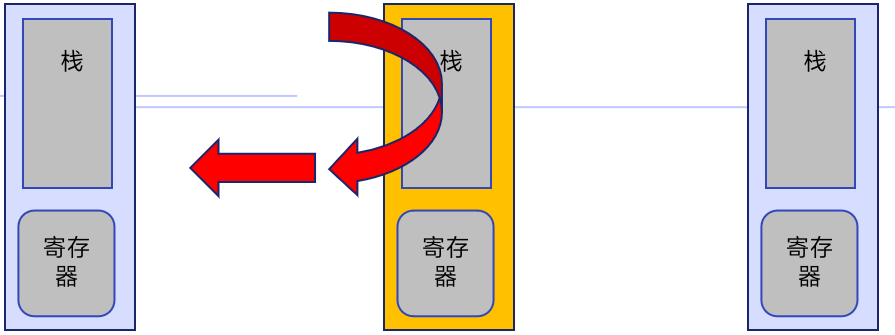
任务切换的实现

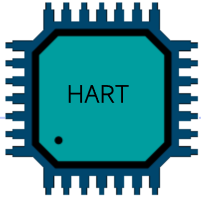

void scheduler(void)
{
struct proc *p;
struct cpu *c = mycpu();
c->proc = 0;
for(;;){
for(p = proc; p < &proc[NPROC]; p++) {
if(p->state == RUNNABLE) {
// Switch to chosen process. It is the process's job
p->state = RUNNING;
c->proc = p;
swtch(&c->context, &p->context);
c->proc = 0;
}
}
}
}
# void swtch(struct context *old,
# struct context *new);
.globl swtch
swtch:
sd ra, 0(a0)
// save ......
sd s11, 104(a0)
ld ra, 0(a1)
// restore ......
ld s11, 104(a1)
ret
jalr x0, 0(ra)
addi a1,s1,16
mv a0,s4
jal ra, <swtch>
// start() jumps here in
// supervisor mode on
void main()
{
// other initializati
procinit();
userinit();
scheduler();
}
p->state = UNUSED;
p->kstack = (uint64)kalloc();
// Set up first user process.
void userinit(void)
{
struct proc *p;
p = allocproc(user_task0);
p->state = RUNNABLE;
}
static struct proc
allocproc(void (*start_routin)(void))
{
// ......
p->state = USED;
// Set up new context to start executing
memset(&p->context, 0, sizeof(p->context));
p->context.ra = (uint64)start_routin;
p->context.sp = p->kstack + PGSIZE;
return p;
}

任务切换的实现

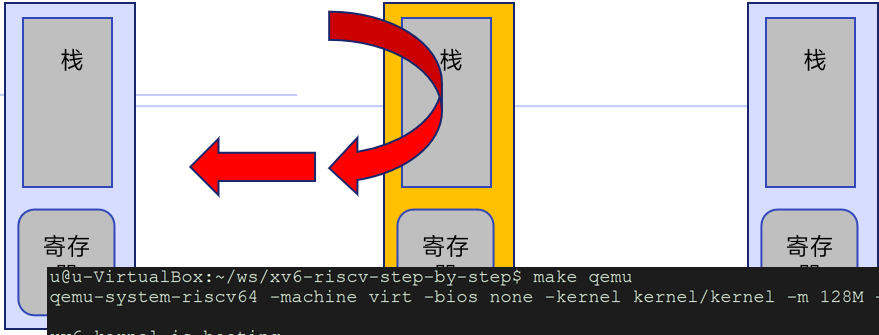

void scheduler(void)
{
struct proc *p;
struct cpu *c = mycpu();
c->proc = 0;
for(;;){
for(p = proc; p < &proc[NPROC]; p++) {
if(p->state == RUNNABLE) {
// Switch to chosen process. It is the process's job
p->state = RUNNING;
c->proc = p;
swtch(&c->context, &p->context);
c->proc = 0;
}
}
}
}
# void swtch(struct context *old,
# struct context *new);
.globl swtch
swtch:
sd ra, 0(a0)
// save ......
sd s11, 104(a0)
ld ra, 0(a1)
// restore ......
ld s11, 104(a1)
ret
jalr x0, 0(ra)
addi a1,s1,16
mv a0,s4
jal ra, <swtch>
// start() jumps here in
// supervisor mode on
void main()
{
// other initializati
procinit();
userinit();
scheduler();
}
p->state = UNUSED;
p->kstack = (uint64)kalloc();
// Set up first user process.
void userinit(void)
{
struct proc *p;
p = allocproc(user_task0);
p->state = RUNNABLE;
}
static struct proc
allocproc(void (*start_routin)(void))
{
// ......
p->state = USED;
// Set up new context to start executing
memset(&p->context, 0, sizeof(p->context));
p->context.ra = (uint64)start_routin;
p->context.sp = p->kstack + PGSIZE;
return p;
}

任务切换的实现



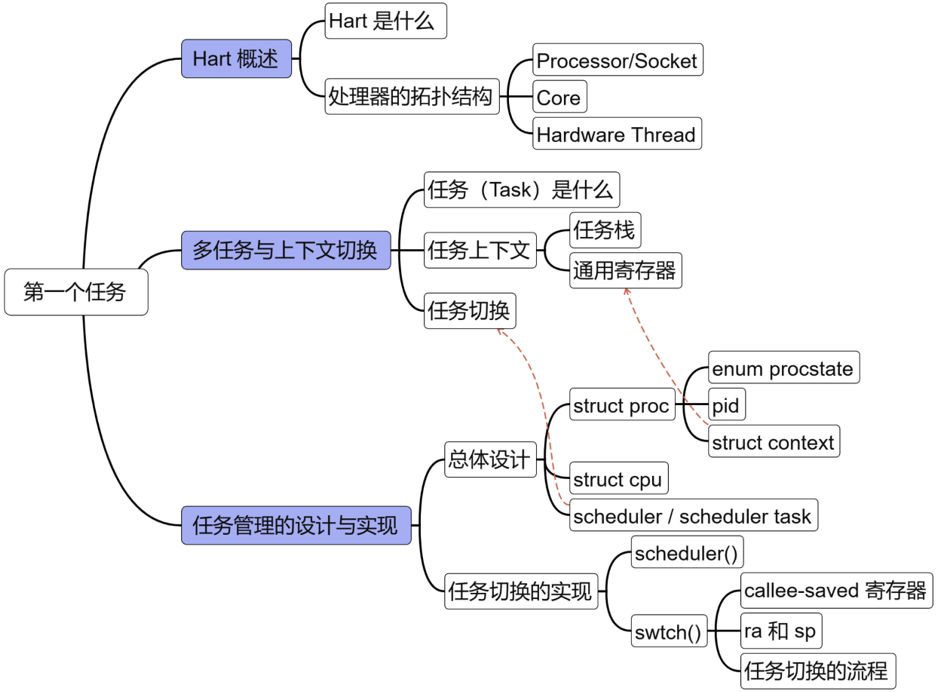


任务切换的实现




```
u@u-VirtualBox:~/ws/xv6-riscv-step-by-step$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic

xv6 kernel is booting

Task 0: Created!
Task 0: Running...
Task 0: Running...
Task 0: Running...
Task 0: Running...
Task 0: Running...
Task 0: Running...
Task 0: Running...
Task 0: Running...
Task 0: Running...
```

本章总结


第一个任务
Hart 概述
Hart 是什么
处理器的拓扑结构
Processor/Socket
Core
Hardware Thread
多任务与上下文切换
任务（Task）是什么
任务上下文
任务栈
通用寄存器
任务切换
任务管理的设计与实现
总体设计
struct proc
enum procstate
pid
struct context
struct cpu
scheduler / scheduler task
任务切换的实现
scheduler()
swtch()
callee-saved 寄存器
ra 和 sp
任务切换的流程

谢谢